

Déployer MLflow sur son propre EC2

Instance dédiée au tracking server MLflow (registre de modèles + artefacts) pour NappeCast avec une image gérée, un seul conteneur, backend SQLite local et artefacts sur S3.

CLÉ AWS STATIQUE À RETIRER EN PRIORITÉ

`docker/mlflow/aws_create_secret.sh` pousse aujourd'hui une paire `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` en clair dans le secret `mlflow/prod-latest` la même clé que celle utilisée par `app_api`. C'est elle qui donne au conteneur MLflow le droit d'écriter les artefacts sur S3 (`--default-artifact-root`). Voir le document « Gestion des IAM » pour la remplacer par un rôle d'instance ; ce guide part du principe que c'est fait.

- 1 Le secret
- 2 Rôle IAM EC2
- 3 Dockerfile
- 4 docker-compose
- 5 deploy.sh
- 6 Workflow CD
- 7 Reverse proxy
- 8 Checklist

1 Secret `mlflow/prod-latest`

Une fois la clé statique retirée (étape 2), ce secret peut redevenir vide ou n'accueillir qu'une future valeur réellement sensible (aucune n'existe aujourd'hui car MLflow lui-même n'a pas d'authentification activée sur le serveur, voir le point de vigilance à l'étape 7).

```
BASH À L'ÉTAT CIBLE, APRÈS RETRAIT DE LA CLÉ STATIQUE
aws secretsmanager put-secret-value \
  --secret-id mlflow/prod-latest \
  --region eu-west-3 \
  --secret-string '{}'
```

Un secret vide reste nécessaire si `deploy.sh` ou `docker-compose.yml` référencent `.env.runtime` car sinon retirez la ligne `get-secret-value` de `deploy.sh` et le fichier `.env.runtime` de l'`env_file` du compose.

2 Rôle IAM de l'instance

Détail complet dans « Gestion des IAM » (étape 3). En résumé, ce rôle doit remplacer la clé statique pour deux usages : lire le secret, et écrire/lire les artefacts sur S3 sous le préfixe utilisé par `MLFLOW_ARTIFACT_ROOT`.

```
JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
```

```

    "Resource": "arn:aws:secretsmanager:eu-west-3:844099234486:secret:mlflow/prod-latest-*"
  },
  { "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*" },
  {
    "Effect": "Allow",
    "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
    "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/mlflow"
  },
  {
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::nappecast", "arn:aws:s3:::nappecast/mlflow-artifacts/*"]
  }
]
}

```

Attachez ce rôle via *EC2* → *Instances* → *Actions* → *Security* → *Modify IAM role* → pas besoin de relancer l'instance. Le SDK `boto3` embarqué dans l'image (voir l'étape 3) utilisera automatiquement les credentials de l'instance d'où que la clé statique est retirée de l'environnement du conteneur.

3 Dockerfile

Image volontairement minimale : Python 3.11-slim, le client MLflow (même version que celle utilisée avec Airflow pour éviter tout conflit de format de log de run), `boto3` pour S3 et `psycopg2-binary` → présent mais actuellement inutilisé puisque le backend store est SQLite (voir l'avertissement à l'étape 4).

```

DOCKERFILE À DOCKER/MLFLOW/DOCKERFILE

FROM python:3.11-slim

RUN pip install --no-cache-dir "mlflow==3.14.0" boto3 psycopg2-binary

EXPOSE 5000

CMD mlflow server \
  --backend-store-uri "$MLFLOW_BACKEND_STORE_URI" \
  --default-artifact-root "$MLFLOW_ARTIFACT_ROOT" \
  --host 0.0.0.0 \
  --port 5000 \
  --workers 1 \
  --allowed-hosts "*" \
  --cors-allowed-origins "$CORS_ALLOWED_ORIGINS"

```

PAS DE --CONSTRAINT ICI, ET C'EST NORMAL

Contrairement à l'image Airflow, MLflow ne publie pas de fichier de contraintes officiel à utiliser avec `pip install` → l'impact de comportement entre les deux Dockerfiles du projet est volontaire, pas un oubli. → pingler `mlflow==3.14.0` explicitement suffit à garantir la reproductibilité du build ici.

4 docker-compose.yml

```
YAML Â€" /OPT/MLFLOW/DOCKER-COMPOSE.YML

services:
  mlflow:
    image: 844099234486.dkr.ecr.eu-west-3.amazonaws.com/mlflow:latest
    ports:
      - "5000:5000"
    volumes:
      - mlflow_data:/mlflow-data
    env_file:
      - .env.runtime
      - mlflow.env
    restart: unless-stopped

volumes:
  mlflow_data:
```

mlflow.env (non-secret, versionn  ) :

```
DOTENV Â€" DOCKER/MLFLOW/MLFLOW.ENV

MLFLOW_TRACKING_URI="http://172.31.32.79:5000/"
MLFLOW_EXPERIMENT_NAME="nappe-cast"
MLFLOW_BACKEND_STORE_URI=sqlite:///mlflow-data/mlflow.db
MLFLOW_ARTIFACT_ROOT=s3://nappecast/mlflow-artifacts
CORS_ALLOWED_ORIGINS=https://mlflow.mapflowing.com
AWS_DEFAULT_REGION=eu-west-3
```

BACKEND STORE SQLITE : LES M  TADONN  ES DE RUNS NE SONT PAS SUR S3

MLFLOW_ARTIFACT_ROOT pointe vers S3 (les gros fichiers Â€" mod  les, plots Â€" sont donc durables), mais MLFLOW_BACKEND_STORE_URI est un fichier SQLite dans le volume Docker nomm   mlflow_data . Ce volume survit    un red  marrage ou une recr  ation du conteneur, **mais pas**    la suppression de l'instance EC2 ou    un remplacement de volume EBS. Toutes les m  tadonn  es de runs (m  triques, param  tres, historique du Model Registry Â€" donc l'info    quelle version est en production   ) seraient perdues dans ce cas, m  me si les artefacts restent sur S3.

Deux options, par ordre de robustesse croissante : (a) sauvegarder p  riodiquement /mlflow-data/mlflow.db vers S3 (un sidecar de sync, comme celui d'Airflow) ; (b) migrer le backend store vers la base RDS Postgres d  j   utilis  e par app_api, en lui donnant son propre sch  ma ou une base d  di  e Â€" psycopg2-binary est d  j   install   dans l'image pour   sa, il ne manque que l'URI (postgresql://  ) dans MLFLOW_BACKEND_STORE_URI et le secret correspondant.

AUCUNE AUTHENTIFICATION SUR LE TRACKING SERVER

--allowed-hosts "*" et l'absence de tout m  canisme d'auth MLflow signifient que quiconque atteint mlflow.mapflowing.com Â€" via l'allowlist IP du reverse proxy (  tape 7) Â€" peut lire, cr  er et supprimer des runs et des mod  les sans identifiant. L'allowlist IP au niveau Caddy est actuellement la seule protection. Si l'acc  s doit un jour s'  largir (nouveaux collaborateurs, IP dynamique), MLflow propose une authentification basique optionnelle (mlflow server --app-name basic-auth)    consid  rer avant d'ouvrir davantage l'acc  s r  seau.

5 deploy.sh

```
BASH Â€" /OPT/MLFLOW/DEPLOY.SH
```

```
#!/usr/bin/env bash
set -euo pipefail

APP_DIR=/opt/mlflow
ENV_FILE="$APP_DIR/.env.runtime"
SECRET_ID="mlflow/prod-latest"
REGION="eu-west-3"
ECR_REGISTRY="844099234486.dkr.ecr.eu-west-3.amazonaws.com"

umask 077
aws secretsmanager get-secret-value \
  --secret-id "$SECRET_ID" --region "$REGION" \
  --query SecretString --output text \
  | jq -r 'to_entries[] | "\(.key)=\(.value)"' > "$ENV_FILE"

chmod 600 "$ENV_FILE"
chown appuser:appuser "$ENV_FILE" "$APP_DIR/docker-compose.yml"

aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

cd "$APP_DIR"
docker compose pull
docker compose --env-file "$ENV_FILE" up -d
docker image prune -f
```

DEUX FICHIERS D'ENVIRONNEMENT, UN SEUL PASSAGE EN `--ENV-FILE`

Le compose (à la tape 4) déclare `env_file: [.env.runtime, mlflow.env]` et les deux alimentent bien les conteneurs. Mais `docker compose --env-file "$ENV_FILE" up -d` ne passe que `.env.runtime`. À l'instar de Compose lui-même (pour résoudre d'éventuels `{VAR}` écrits en dur dans le YAML et il n'y en a aucun ici, donc sans conséquence actuellement). Le jour où quelqu'un ajoute un `{VAR}` non secret dans ce compose (par exemple pour un tag d'image), il faudra passer `--env-file mlflow.env` à la place, ou lister les deux fichiers séparés par une virgule si votre version de Compose le permet.

6 Workflow CD (`cd-mlflow.yml`)

Structure identique à Airflow : déclenché sur `docker/mlflow/**`, build conditionné à l'inexistence du tag, ouverture temporaire du port 22, SCP puis SSH.

Secret GitHub

`MLFLOW_SG_ID`

`MLFLOW_EC2_HOST` / `_USER` / `_SSH_KEY`

Usage

Security group dont le port 22 est ouvert/fermé le temps du déploiement

Connexion SCP/SSH à cette instance précise

`command_timeout: 30m` est déclaré présent sur l'écoulement SSH et utile ici aussi, l'image de base `python:3.11-slim` restant plus légère que celle d'Airflow, le premier `pull` devrait rarement l'atteindre.

7 Reverse proxy et pas sur cette instance

MLflow n'a pas de TLS ni de contrôle d'accès lui-même : c'est le Caddy tournant sur

l'**instance app_api** qui termine le TLS pour `mlflow.mapflowing.com` et relaie en HTTP simple vers l'IP privée de cette instance sur le port 5000, avec une allowlist d'IP. Voir le document `app_api`, [tâche 5](#), pour le détail du `Caddyfile`. Ça veut dire concrètement :

- ✓ Le security group de **cette** instance doit autoriser le port `5000` en entrant uniquement depuis l'IP privée (VPC) de l'instance `app_api` – jamais depuis Internet.
- ✓ Si l'IP privée de cette instance change (redémarrage avec une IP non-Elastic, remplacement d'instance), le `Caddyfile` de `app_api` doit être mis à jour – pensez à une IP privée Elastic ou une entrée DNS interne si ce genre de changement devient fréquent.

8 Checklist

- ✓ **Règles IAM d'instance en place**, clôt statique retirée du secret et de l'environnement des conteneurs.
- ✓ **Stratégie de sauvegarde du backend store SQLite** déployée et en place (sync S3 ou migration Postgres).
- ✓ **Port 5000 restreint** à l'IP privée de l'instance `app_api` uniquement.
- ✓ **Authentification MLflow** évaluée si l'accès doit s'élargir au-delà de l'allowlist IP actuelle.
- ✓ **Volume EBS dimensionné** pour la croissance du SQLite et de tout cache local, même si les artefacts eux-mêmes vivent sur S3.

Voir « Gestion des IAM » pour le détail des règles, « Pipeline Secrets AWS » pour le mécanisme générique, et « Déploiement `app_api` » pour le reverse proxy qui expose ce service au monde extérieur.